

Python Tip: Sets

Need to remove duplicate values from a list?

Want to compare two groups of values?

Python sets make both tasks easy!



What Is a Set?

A set is similar to a list, but every value can appear only once. Its values also do not have fixed positions.

You can create a set with curly braces or by passing another collection to `set()`.

```
colors_a = {"red", "blue"}  
colors_b = set(["red", "blue"])  
  
empty_colors = set()
```

An empty set must be written as `set()` because `{}` creates an empty dictionary instead.

A Common Use: Remove Duplicates

Because a set keeps only one copy of each value, it can remove duplicate values from a list.

Imagine a signup list where some names were entered more than once. We can convert the list to a set and then back to a list.

```
signups = [  
    "alice", "bob", "alice",  
    "carol", "bob"  
]  
  
signup_set = set(signups)      # List to set  
unique_signups = list(signup_set) # Set to list  
  
print("Entries:", len(signups))  
print("People:", len(unique_signups))
```

```
Entries: 5  
People: 3
```

We started with five entries. After removing the duplicates, the new list contains three names.

Add and Check Values

Once you have a set, use `.add()` to add another value. Use `in` to check whether a value is in the set.

```
members = {"alice", "bob"}

members.add("carol")
members.add("carol") # Doesn't do anything

print("Members:", len(members))
print("Carol:", "carol" in members)
print("Dave:", "dave" in members)
```

```
Members: 3
Carol: True
Dave: False
```

Carol was added twice, but the set still contains only three members.

Compare Sets

Sets become especially useful when you want to compare groups.

The **intersection** (`&`) includes values in both sets.

The **union** (`|`) includes all values from both sets.

The **difference** (`-`) includes values from the first set that are not in the second.

```
designers = {"alice", "bob"}  
developers = {"bob", "carol"}  
  
print("Intersection:", designers & developers)  
print("Union:", designers | developers)  
print("Difference:", designers - developers)
```

```
Intersection: {'bob'}  
Union: {'alice', 'bob', 'carol'}  
Difference: {'alice'}
```

Sets do not use `+`. To combine two sets, use the union operator (`|`).

A Common Use: Find Missing Values

Set difference is commonly used to check for missing or unexpected values.

Imagine Alice, Bob, and Carol are expected at an event, but Bob, Carol, and Dave arrive.

```
expected = {"alice", "bob", "carol"}  
arrived = {"bob", "carol", "dave"}  
  
missing = expected - arrived  
unexpected = arrived - expected  
  
print("Missing:", missing)  
print("Unexpected:", unexpected)
```

```
Missing: {'alice'}  
Unexpected: {'dave'}
```

The order matters. `expected - arrived` gives us the missing names. Reversing the order gives us the unexpected names.

WARNING: Sets Do Not Preserve Order

A set does not keep track of the original positions of its values. Run this script once, then run the whole script again.

```
letters = ["a", "b", "c", "d", "e", "a"]  
  
print(list(set(letters)))  
  
# Run 1:  
# ['e', 'c', 'd', 'b', 'a']  
  
# Run 2:  
# ['a', 'e', 'b', 'c', 'd']
```

The duplicate is removed both times, but the order may change between program runs. Use `sorted(set(...))` when you need a consistent, sorted order.

Wrap-Up

Sets keep one copy of each value and make it easy to compare groups.

Use `&` for an intersection, `|` for a union, and `-` for a difference.

Use a list instead when order or duplicate values matter.

Follow me for more tips.

- Shep4